

Programação Orientada a Objetos

**Encapsulamento, Métodos GET e SET, modificadores de acesso em
JAVA**

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



- 1 Introdução Histórica
- 2 Encapsulamento: Definição e Importância
- 3 Encapsulamento
- 4 Método GET
- 5 Método SET

**Quando você vai se mudar,
o que você costuma fazer
com suas coisas?**

O Livro: A Invenção do Container



Figura: Livro "The Box" de Marc Levinson

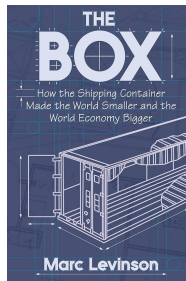


Figura: Livro "The Box" de Marc Levinson

Obra

The Box: How the Shipping Container Made the World Smaller and the World Economy Bigger

Autor: Marc Levinson Ano de publicação: 2006



Transporte Antes da Containerização



Figura: Transporte global antes da padronização dos contêineres

Transporte Antes da Containerização

Contexto Histórico

Antes da padronização, as mercadorias eram transportadas manualmente entre armazéns, navios e trens, gerando lentidão e altos custos operacionais.



Exemplo: 100.000 bicicletas transportadas uma a uma

Problema Logístico

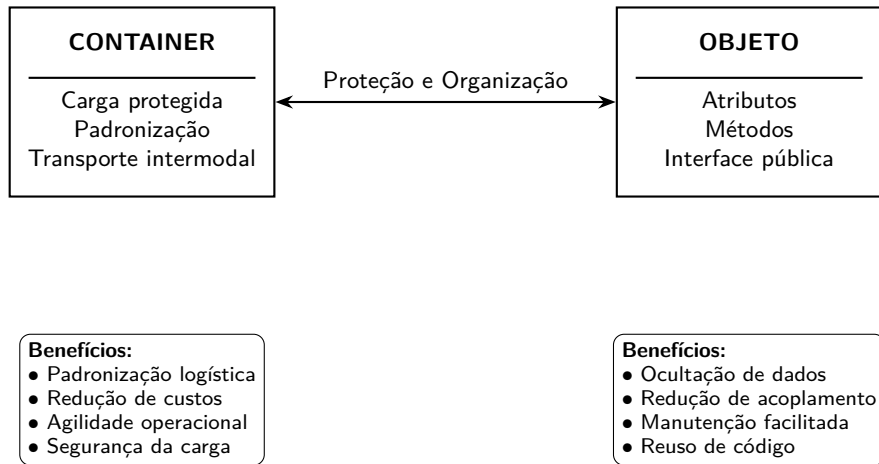
- Falta de padronização
- Processo lento e custoso
- Elevado esforço humano
- Gargalos nos portos

Tema Central

A obra analisa como a padronização do transporte por contêiner transformou radicalmente:

- O comércio internacional
- A logística global
- A organização das cadeias produtivas

Container vs Encapsulamento em Objetos



Analogia Estrutural

O container encapsula mercadorias para transporte eficiente. O objeto encapsula dados e comportamentos para organização e segurança do sistema.

Definição

O **Encapsulamento** é um dos pilares da Orientação a Objetos e consiste no **princípio de ocultar os detalhes internos de implementação de uma classe**, permitindo acesso controlado aos seus dados.

Objetivo

- Proteger o estado interno do objeto
- Garantir integridade dos dados
- Controlar como os atributos são modificados

Ideia Central

O acesso direto aos atributos deve ser evitado, sendo realizado por meio de métodos públicos controlados.

Principais Modificadores

- **public** → acesso livre
- **private** → acesso restrito à própria classe
- **protected** → acesso na mesma classe, pacote ou subclasses
- **default** → acesso apenas dentro do pacote

Encapsulamento na Prática

Para aplicar encapsulamento:

- Declarar atributos como **private**
- Criar métodos públicos de acesso

Modificador `public`

Onde é aplicado

Classes, atributos, métodos e construtores.

Função

Permite acesso a partir de qualquer classe, independentemente do pacote.

Uso típico

Define a interface pública do objeto.

```
public class Camera {  
  
    public String marca;  
  
    public void ligar() {  
        System.out.println("Ligando camera");  
    }  
}
```

```
Camera c = new Camera();  
  
c.marca = "Canon";      // POSSÍVEL  
c.ligar();               // POSSÍVEL
```

Observação

Pode ser acessado de qualquer classe.

Modificador `private`

Onde é aplicado

Atributos, métodos e construtores.

Função

Restringe o acesso exclusivamente à própria classe.

Objetivo

Implementar encapsulamento e proteção de estado.

```
public class Camera {  
  
    private String modelo;  
  
    private void resetar() {  
        System.out.println("Resetando...");  
    }  
  
    public String getModelo() {  
        return modelo;  
    }  
}
```

```
Camera c = new Camera();  
  
c.modelo = "X100";           // IMPOSSÍVEL  
c.resetar();                 // IMPOSSÍVEL  
c.getModelo();               // POSSÍVEL
```

Observação

Acesso somente por métodos públicos da própria classe.

Onde é aplicado

Atributos, métodos e construtores.

Função

Permite acesso:

- Na própria classe
- No mesmo pacote
- Em subclasses (mesmo em pacotes diferentes)

Onde é aplicado

Atributos, métodos e construtores.

Função

Permite acesso:

- Na própria classe
- No mesmo pacote
- Em subclasses (mesmo em pacotes diferentes)

```
Camera c = new Camera();  
  
c.resolucao = 20;    // IMPOSSÍVEL fora do pacote  
c.configurar();     // IMPOSSÍVEL fora do pacote
```

Observação

Acesso permitido em subclasses e mesmo pacote.

Modificador default

Onde é aplicado

Classes (sem modificador), atributos e métodos.

Função

Permite acesso apenas dentro do mesmo pacote.

Importante

É o modificador padrão quando nenhum é declarado.

```
class Camera {  
  
    String modelo;        // default  
  
    void ligar() {        // default  
        System.out.println("Ligando...");  
    }  
}
```

```
class Camera {  
  
    String modelo;        // default  
  
    void ligar() {        // default  
        System.out.println("Ligando...");  
    }  
}
```

```
public class ContaBancaria {  
    public double saldo;  
}
```

Problema

O atributo saldo pode ser alterado diretamente:

```
conta.saldo = -1000;
```

Isso compromete a integridade do objeto.

```
public class ContaBancaria {  
  
    private double saldo;  
  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
        }  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
}
```

Análise Técnica

- O atributo saldo é privado
- A modificação ocorre apenas por métodos controlados
- Regras de negócio podem ser implementadas

Classe cameraDigital

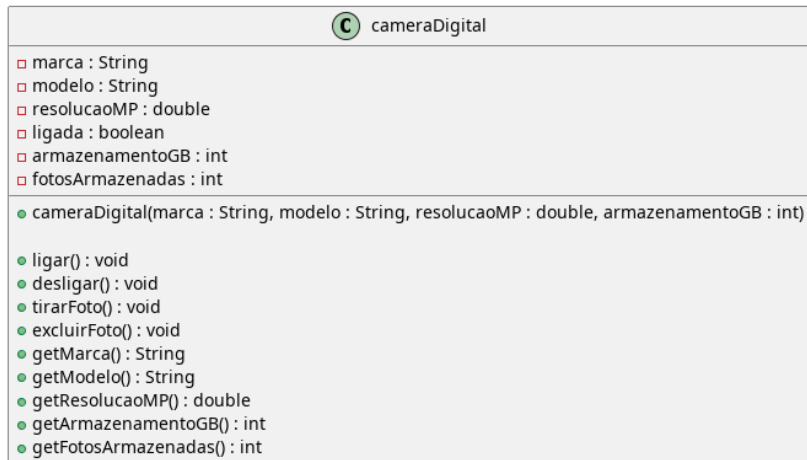


Figura: Diagrama de classe da classe CameraDigital

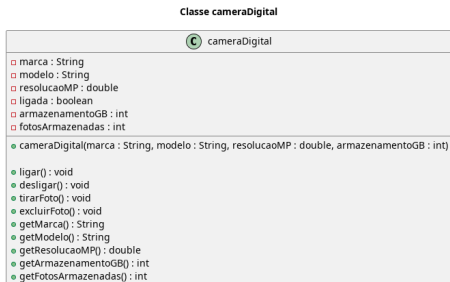


Figura: Diagrama de classe da classe CameraDigital

Análise Técnica

- Atributos privados: `resolucao`, `zoom`, `ligada`
- Métodos públicos: `tirarFoto()`, `ligar()`, `desligar()`
- Encapsulamento protege o estado interno da câmera
- Interface pública controla o comportamento da câmera.

Método GET

Onde é aplicado

- Em classes que utilizam encapsulamento
- Para acessar atributos privados

Função

- Retornar o valor de um atributo
- Permitir leitura controlada do estado do objeto

Características

- Geralmente público
- Não recebe parâmetros
- Retorna o tipo do atributo

```
public class Camera {  
  
    private String modelo;  
  
    public String getModelo() {  
        return modelo;  
    }  
}
```

Invocação

```
Camera c = new Camera();  
String m = c.getModelo(); // leitura permitida
```

Onde é aplicado

- Em classes com atributos privados
- Para modificar valores internos

Função

- Alterar o valor de um atributo
- Permitir escrita controlada

Características

- Geralmente público
- Recebe um parâmetro
- Retorna void

Exemplo de Método SET com Validação

```
public class Camera {  
  
    private String modelo;  
  
    public void setModelo(String modelo) {  
        if (modelo == null || modelo.trim().isEmpty()) {  
            throw new IllegalArgumentException(  
                "O modelo não pode ser vazio."  
            );  
        }  
        this.modelo = modelo;  
    }  
}
```

Invocação

```
Camera c = new Camera();  
c.setModelo("Canon");    // válido  
  
c.setModelo("");         // lança exceção  
c.setModelo(null);       // lança exceção
```



Figura: Cafeteira como exemplo de encapsulamento e café como commodity transportável.